E-Mall Backend API Documentation

E-Mall Backend API Documentation

Generated from the live Express router, middleware, validation, controller, service, and Mongoose model files in `src/`.

- **Application root:** `src/app.ts`
- **Framework:** Express 5.2.1, TypeScript, Mongoose 9.1.5
- **Validation:** Zod 4.3.6 via `src/middlewares/validate-body.middleware.ts` and selected controller-level `schema.parse(...)`
- **Authentication:** JWT bearer token via `src/middlewares/authentication.ts`
- **Global middleware:** `express.json`, `cookie-parser`, XSS sanitizer, Helmet, `quickValidate`
- **Base URL:** not hard-coded in the project; examples use `http://localhost:<PORT>`

Table of Contents

Global API Behavior

Authentication

Protected endpoints require:

Header	Required	Example	Notes
`Authorization`	Yes	`Bearer eyJhbGciOi...`	Parsed by `authentication.ts`. The token payload must include `id`; the user must still exist.
`Content-Type`	For JSON bodies	`application/json`	All body validators expect JSON.
`Cookie`	For refresh/logout	`refreshToken=<token>`	Refresh tokens are stored in an HTTP-only `refreshToken` cookie.

Authentication error examples:

```
{
  "status": "fail",
  "message": "You are not logged in. Please log in to get access"
}
```

```
{
  "status": "fail",
```

```
  "message": "Your session has expired. Please log in again."
}
```

Roles

The application defines these roles in `src/apis/auth/models/user.model.ts`:

Role	Usage
`admin`	Category administration, store administration, store management, product deletion, delivery operations.
`owner`	Store/product/order management for owned stores, product deletion.
`user`	Default registered customer role.
`accounting`	Defined but not used by the current routers.
`fulfillment`	Delivery/order fulfillment routes.

Standard Error Shape

Operational errors are produced by `AppError` and handled by `global-error-handling.middleware.ts`.

```
{
  "status": "fail",
  "message": "Validation or business error message"
}
```

Common error codes:

Status	Meaning	Typical Causes
400	Bad request	Zod validation failure, invalid ID, invalid state, duplicate value.
401	Unauthorized	Missing/invalid/expired JWT, missing refresh cookie.
403	Forbidden	Valid user lacks required role or

Status	Meaning	Typical Causes
		ownership.
404	Not found	Missing user, category, store, product, address, cart, wishlist, or order.
409	Conflict	Invalid order state transition or SKU uniqueness conflict.
429	Too many requests	Rate limiter on `/auth/refresh`.
500	Server error	Unexpected runtime/database error.

Global Validation and Sanitization

`quickValidate` runs before all routes and validates/sanitizes common fields if present:

Field Location	Field	Rules
Body	`email`	Valid email, trimmed, lowercased.
Query	`email`	Valid email, trimmed, lowercased.
Params	`email`	Valid email, trimmed, lowercased.
Body	`password`	Min 8, must contain English letters, a number, and a symbol.
Params/Query/Body	`token`	Exactly 64 hex characters.

`xss.ts` recursively sanitizes string values in `req.body`, `req.query`, and `req.params`.

Data Models and Relations

Model	File	Main F
User	src/apis/auth/models/user.model.ts	email, name, phoneNumber, isActive, hashPasswor verification fl
LoginSession	src/apis/auth/models/loginSession.model.ts	userId, hash refreshToke userAgent, re expiresAt
VerifyTokens	src/apis/auth/models/verify-tokens.model.ts	userId, toke expiresAt, us verifiedAt
Category	src/apis/category/models/category.model.ts	name, slug, pa
Store	src/apis/store/models/store.model.ts	name, logo, ov categoryId, authorizedB isActive, opening/clos deletedAt
Product	src/apis/products-and-variants/model/products-and-variants.model.ts	name, slug, description, categoryId, variants, tag isActive, sa
Cart	src/apis/cart/models/cart.model.ts	userId, item

Model	File	Main F
WishList	src/apis/wish-list/models/wish-list.model.ts	userId, items
Address	src/apis/address/model/address.model.ts	userId, stree distanceMark notes, isDefa
Order	src/apis/order/models/order.model.ts	buyer snapsh slices, missin order/payme status, totals

Auth Module

Router file: src/apis/auth/router/auth.route.ts

Auth Request Schemas

Schema	Field	Type	Required	Va
RegisterBody	name	string	Yes	Trir 1, n
RegisterBody	email	string	Yes	Vali trin low
RegisterBody	password	string	Yes	Mir con nur sym
RegisterBody	phone	string	No	Reg ^01

Schema	Field	Type	Required	Va
LoginBody	email	string	Yes	Vali low
LoginBody	password	string	Yes	Mir 128
ForgotPasswordBody	email	string	Yes	Vali low
ResetPasswordBody	newPassword	string	Yes	Cor req glol pas app fiel pas this che con mat
ResetPasswordBody	confirmPassword	string	Yes	Mu new
EmailVerificationBody	email	string	Yes	Vali glol qui
EmailVerificationBody	verifyCode	string	Yes	Req con
SendEmailVerificationCodeBody	email	string	Yes	Vali glol qui

Auth Endpoints

Method	Full Route Path	Short Description	Controller / Handler
POST	/auth/register	Create a user, cart, wishlist, login session,	register

Method	Full Route Path	Short Description	Controller / Handler
		access token, and refresh cookie.	
POST	/auth/login	Authenticate with email/password and create a refresh session.	login
POST	/auth/refresh	Rotate refresh token cookie and return a new access token.	refreshToken
GET	/auth/logout	Revoke current refresh cookie session and clear cookie.	logout
POST	/auth/forgot-password	Create a password-reset token and send email.	forgotPassword
POST	/auth/reset-password/:token	Reset password using a 64-character reset token.	resetPassword
POST	/auth/send-code-email-verification	Send an email verification code.	sendCodeEmailVerification
PATCH	/auth/email-verification	Verify email using email and verification code.	emailVerification

Auth Endpoint Details

Endpoint	Authentication	Roles / Permissions	Required Headers	P
`POST /auth/register`	No	Public	`Content-Type: application/json`	Noı
`POST /auth/login`	No	Public	`Content-Type: application/json`	Noı
`POST /auth/refresh`	Refresh cookie required	Public session	Cookie: `refreshToken`	Noı
`GET /auth/logout`	Refresh cookie optional	Current session	Cookie: `refreshToken` if present	Noı
`POST /auth/forgot-password`	No	Public	`Content-Type: application/json`	Noı
`POST /auth/reset-password/:token`	No	Reset token	`Content-Type: application/json`	`tok` stri req hex exa 5a2
`POST /auth/send-code-email-verification`	No	Public	`Content-Type: application/json`	Noı

Endpoint	Authentication	Roles / Permissions	Required Headers	P
PATCH /auth/email-verification	No	Public	Content-Type: application/json	Noı

Example register request:

```
{
  "name": "Mohammed Hassan",
  "email": "user@example.com",
  "password": "StrOng!Pass",
  "phone": "01012345678"
}
```

Example auth response:

```
{
  "status": "success",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "data": {
    "user": {
      "id": "665f51b9e5348a2f42b56c41",
      "name": "Mohammed Hassan",
      "email": "user@example.com",
      "phoneNumber": "01012345678"
    }
  }
}
```

Categories Module

Router file: `src/apis/category/router/category.route.ts`

Category Request Schemas

Schema	Field	Type	Required	Valid / R
CreateCategoryBody	name	string	Yes	Trim min 1 100
CreateCategoryBody	parentId	ObjectId string	No	Must valid Mong Objec trans to Ob
UpdateCategoryBody	name	string	No	Trim min 1 100
UpdateCategoryBody	parentId	ObjectId string	No	Valid Objec
ReplaceCategoryBody	replacementCategoryId	ObjectId string	Yes	Valid Objec
ReplaceCategoryBody	reason	string	No	Trim max 5

Category Endpoints

Endpoint	Description	Handler	
GET /categories/	List all categories.	listCategories	src/
GET /categories/:id/usage	Check if a category is used by stores/products.	getCategoryUsage	same
GET /categories/:id	Fetch one category by ID.	getCategory	same

Endpoint	Description	Handler	
`POST /categories/`	Create a category.	`createCategory`	same
`PATCH /categories/:id`	Update name or parent.	`updateCategoryHandler`	same
`POST /categories/:id/replace`	Validate replacement and return replacement-processing result.	`replaceCategoryHandler`	same
`DELETE /categories/:id`	Delete an unused category.	`deleteCategory`	same

Example category response:

```
{
  "status": "success",
  "data": {
    "category": {
      "_id": "665f51b9e5348a2f42b56c41",
      "name": "Electronics",
      "slug": "electronics",
      "parentId": null,
      "createdAt": "2026-05-10T00:00:00.000Z",
      "updatedAt": "2026-05-10T00:00:00.000Z"
    }
  }
}
```

Stores Module

Router file: `src/apis/store/router/store.router.ts`

Store Request Schemas

Schema	Field	Type	Required	Validation Rules
`CreateStoreBody`	`name`	string	Yes	Trimmed max 100
`CreateStoreBody`	`email`	string	Yes	Valid email lowercase
`CreateStoreBody`	`categoryName`	string	Conditional	Required `category` absent; mutually exclusive `category`
`CreateStoreBody`	`categoryId`	ObjectId string	Conditional	Required `category` absent; valid ObjectId; mutually exclusive `category`
`CreateStoreBody`	`authorizedBrand`	string	No	Trimmed max 100
`CreateStoreBody`	`logo`	string	No	Trimmed max 100
`CreateStoreBody`	`openingTime`	string	No	Exactly 5 intended
`CreateStoreBody`	`closingTime`	string	No	Exactly 5 intended
`UpdateStoreOrderBody`	`status`	enum	Yes	`pending`, `preparing` `ready`, `re`
`UpdateStoreOrderBody`	`rejectionReason`	string	Conditional	Required rejected

Schema	Field	Type	Required	Valida Rul
				missingI provided
UpdateStoreOrderBody	missingItems[]	array	Conditional	Required rejected u rejectio provided
StoreProductBody	name	string	Yes on create	Min 1
StoreProductBody	slug	string	No	Min 1
StoreProductBody	description	string	No	Any strin
StoreProductBody	brand	string	No	Any strin
StoreProductBody	categoryId	string	Yes on create	Min 1; se verifies c
StoreProductBody	basePrice	number	No	Min 0
StoreProductBody	variants[]	array	Yes on create	Min 1 on
StoreProductBody	defaultVariantId	string	No	Must refe one varia supplied
StoreProductBody	images[]	string[]	No	Array of s
StoreProductBody	tags[]	string[]	No	Array of s
StoreProductBody	isActive	boolean	No, update only	Toggle pr visibility
Variant	sku	string	Yes	Min 1; ur store/pro
Variant	price	number	Yes	Min 0
Variant	salePrice	number	No	Min 0, <= requires s window
Variant	saleStartAt	date	Conditional	Required salePric

Schema	Field	Type	Required	Valida Rul
				coerced c
Variant	saleEndAt	date	Conditional	Required salePric must be a start
Variant	imageUrl	string	No	Any strin
Variant	images[]	string[]	No	Array of s
Variant	isDefault	boolean	No	Preferred variant
Variant	attributes[]	array	No	Each iten required and valu

Store Endpoints

Endpoint	Description	
GET /store/	List active stores for public users.	listAllSto
GET /store/category/:categoryId	List active stores in a category.	listStores
GET /store/admin	Admin store listing.	listAllSto

Endpoint	Description	
`GET /store/:storeId`	Fetch a public store by ID.	`getStoreBy`
`GET /store/:storeId/products`	List public active products for a store.	`listStoreP`
`GET /store/:storeId/products/:productId`	Fetch one public store product.	`getStorePr`
`POST /store/`	Create a store for an owner email/category.	`createStor`
`GET /store/my-store`	Get the current owner’s store.	`getMyStore`
`PATCH /store/:storeId/manage/settings`	Update store settings.	`updateMySt`
`GET /store/:storeId/manage/dashboard`	Get store dashboard stats.	`getDashboa`

Endpoint	Description	
`DELETE /store/:storeId/manage/delete`	Permanently delete store.	`hardDelete`
`GET /store/:storeId/manage/products`	List owner/admin products, including inactive when requested.	`listMyStor`
`GET /store/:storeId/manage/products/:productId`	Fetch owner/admin product detail.	`getMyStore`
`POST /store/:storeId/manage/products`	Create product in a store.	`createStor`
`PATCH /store/:storeId/manage/products/:productId`	Update product or variants.	`updateMySt`
`DELETE /store/:storeId/manage/products/:productId`	Delete a store product.	`deleteMySt`
`GET /store/:storeId/manage/orders`	List store orders.	`getMyOrder`

Endpoint	Description	
`GET /store/:storeId/manage/orders/:orderId`	Fetch a store-scoped order.	`getMyOrder`
`PATCH /store/:storeId/manage/orders/:orderId`	Update store order status.	`updateMyOr`

Example store product create request:

```
{
  "name": "iPhone 15",
  "description": "128GB smartphone",
  "brand": "Apple",
  "categoryId": "665f51b9e5348a2f42b56c41",
  "basePrice": 799,
  "variants": [
    {
      "sku": "IPH15-BLK-128",
      "price": 799,
      "salePrice": 749,
      "saleStartAt": "2026-05-10T00:00:00.000Z",
      "saleEndAt": "2026-05-20T00:00:00.000Z",
      "isDefault": true,
      "attributes": [
        { "name": "color", "value": "black" },
        { "name": "storage", "value": "128GB" }
      ]
    }
  ],
  "tags": ["phone", "ios"]
}
```

Example paginated product list response:

```
{
  "status": "success",
```

```
  "data": {
    "products": [
      {
        "_id": "665f51b9e5348a2f42b56c50",
        "name": "iPhone 15",
        "slug": "iphone-15",
        "brand": "Apple",
        "mainVariant": {
          "_id": "665f51b9e5348a2f42b56c51",
          "sku": "IPH15-BLK-128",
          "price": 799,
          "currentPrice": 749,
          "isSaleActive": true
        },
        "category": { "_id": "665f51b9e5348a2f42b56c41", "name":
        "Electronics", "slug": "electronics" },
        "store": { "_id": "665f51b9e5348a2f42b56c52", "name": "Tech
        Hub", "logo": "https://cdn.example.com/logo.png" }
      }
    ],
    "total": 1,
    "page": 1,
    "limit": 24
  }
}
```


Products Module

Router file: `src/apis/products-and-variants/router/product.router.ts`

Product Query Parameters

Name	Type	Required	Default	Description	Exampl
`storeId`	ObjectId string	No	None	Limit products to one store.	`665f51b9e5348a2`
`shopId`	ObjectId string	No	None	Alias for `storeId`.	`665f51b9e5348a2`
`category`	string	No	None	Category ObjectId or	`electronics`

Name	Type	Required	Default	Description	Exampl
				category slug.	
`search`	string	No	None	Case-insensitive match on product name or description.	`iphone`
`minPrice`	number	No	None	Minimum variant price.	`100`
`maxPrice`	number	No	None	Maximum variant price.	`1000`
`sort`	enum	No	`newest` behavior	`price-asc`, `price-desc`, `newest`, `best-selling`	`price-asc`
`page`	integer	No	`1`	Page number.	`2`
`limit`	integer	No	`24`	Page size, max 100.	`20`
dynamic attributes	string	No	None	Any extra query key filters variant attributes; comma values supported.	`color=black,whi`

Product Endpoints

Endpoint	Description	Handler	File
`GET /products`	Search/filter/sort paginated active products.	`listProducts`	`src/apis/products-variants/controll products.controll`

Endpoint	Description	Handler	File
`GET /products/:slug`	Fetch active product by slug; optional store disambiguation.	`getProductBySlug`	`src/apis/products-variants/controll product-by-slug.controller.t`
`DELETE /products/:id`	Soft-delete product by ID.	`deleteProduct`	`src/apis/products-variants/controll product.controlle`

Cart Module

Router file: `src/apis/cart/router/cart.router.ts`

All cart endpoints use `cartRouter.use(authorized)` and require a bearer token.

Cart Request Schemas

Schema	Field	Type	Required	Validation / Rules	Defa
`AddCartItemBody`	`variantId`	string	Yes	Min 1; service must find variant	None
`AddCartItemBody`	`quantity`	integer	No	Min 1	Servi defau 1

Schema	Field	Type	Required	Validation / Rules	Defa
DecreaseCartItemBody	quantity	integer	No	Min 1	Servi defau 1

Cart Endpoints

Endpoint	Description	Handler	
GET /cart/	Get current user's cart.	getCart	src/a cart.
POST /cart/item	Add a product variant to cart.	addCartItem	src/a item.
PATCH /cart/items/:variantId/decrease	Decrease quantity or remove if quantity reaches zero.	decreaseCartItem	src/a cart-
DELETE /cart/item/:variantId	Remove an item by variant.	removeCartItem	src/a cart-
DELETE /cart/	Clear the current user's cart.	clearCart	src/a cart.

Example cart response:

```
{
  "status": "success",
```

```
  "data": {
    "_id": "665f51b9e5348a2f42b56c60",
    "items": [
      {
        "variantId": "665f51b9e5348a2f42b56c51",
        "quantity": 2,
        "product": { "_id": "665f51b9e5348a2f42b56c50", "name":
        "iPhone 15", "slug": "iphone-15" },
        "store": { "_id": "665f51b9e5348a2f42b56c52", "name": "Tech
        Hub", "logo": "https://cdn.example.com/logo.png" }
      }
    ]
  }
}
```


Wishlist Module

Router file: `src/apis/wish-list/router/wish-list.router.ts`

All wishlist endpoints use `wishRouter.use(authorized)` and require a bearer token.

Wishlist Endpoints

Endpoint	Description	Handler	Fil
`GET /wishlist/`	Get current user's wishlist.	`getWishList`	`src/apis/wish list/controll wishlist.cont`
`POST /wishlist/item`	Add a variant to wishlist.	`addWishItem`	`src/apis/wish list/controll wishlist-item.controll`
`DELETE /wishlist/item/:variantId`	Remove a variant from wishlist.	`removeWishItem`	`src/apis/wish list/controll wishlist.cont`

Endpoint	Description	Handler	Fil
`DELETE /wishlist/`	Clear current user's wishlist.	`clearWishList`	`src/apis/wish list/controll wishlist.cont`

Example add wishlist request:

```
{
  "variantId": "665f51b9e5348a2f42b56c51"
}
```

Addresses Module

Router file: `src/apis/address/router/address.router.ts`

All address endpoints use `addressRouter.use(authorized)`.

Address Request Schemas

Schema	Field	Type	Required	Validation / Rules
`CreateAddressBody`	`street`	string	Yes	Trimmed, min 1
`CreateAddressBody`	`city`	string	Yes	Trimmed, min 1
`CreateAddressBody`	`distanceMark`	string	Yes	Trimmed, min 1
`CreateAddressBody`	`phone`	string	Yes	Trimmed, min 1
`CreateAddressBody`	`notes`	string	No	Trimmed
`CreateAddressBody`	`isDefault`	boolean	No	Optional

Schema	Field	Type	Required	Validation / Rules
`UpdateAddressBody`	any address field except `isDefault`	string	At least one	Same string trim rules
`SetDefaultAddressBody`	`isDefault`	boolean	No	Optional; service receives value

Address Endpoints

Endpoint	Description	Handler	
`GET /addresses/`	List current user's addresses.	`getMyAddresses`	`src/apis/addr addresses.cor`
`GET /addresses/default`	Get default address.	`getDefaultAddress`	`src/apis/addr default-addre`
`GET /addresses/:id`	Get one owned address.	`getAddressById`	`src/apis/addr address-by-ic`
`POST /addresses/`	Create address.	`createAddress`	`src/apis/addr address.contr`
`PATCH /addresses/:id`	Update owned	`updateAddress`	`src/apis/addr address.contr`

Endpoint	Description	Handler	
	address.		
`PATCH /addresses/:id/default`	Set address as default.	`setDefaultAddress`	`src/apis/addr default-addre`
`DELETE /addresses/:id`	Delete owned address.	`deleteAddress`	`src/apis/addr address.contr`

Example address response:

```
{
  "status": "success",
  "data": {
    "address": {
      "_id": "665f51b9e5348a2f42b56c70",
      "street": "15 Nile St",
      "city": "Cairo",
      "distanceMark": "Near mall entrance",
      "phone": "01012345678",
      "notes": "Call on arrival",
      "isDefault": true
    }
  }
}
```

Orders Module

Router file: `src/apis/order/router/order.router.ts`

All order endpoints use `orderRouter.use(authorized)`.

Order Request Schemas

Schema	Field	Type	Required	Validation / Rules	Defaul
`CreateOrderBody`	`addressId`	string	Yes	Trimmed, min 1; service validates address ownership	None
`CreateOrderBody`	`paymentMethod`	enum	Yes	`cod`, `online`	None

Order Endpoints

Endpoint	Description	Handler	
`GET /orders/`	List current user's orders.	`getUserOrdersHandler`	sr us
`GET /orders/:orderId`	Fetch one current-user order.	`getUserOrderHandler`	sr us
`POST /orders/`	Create order from current cart and selected address.	`createOrderHandler`	sr or

Endpoint	Description	Handler	
`PATCH /orders/:orderId/accept-partial`	Accept an order after store-side missing items decision.	`acceptPartialOrderHandler`	sr pa
`PATCH /orders/:orderId/cancel`	Cancel current-user order if still cancellable.	`cancelOrderHandler`	sr or
`POST /orders/:orderId/pay`	Create a payment intent for an online order.	`createPaymentIntentHandler`	sr pa

Example create order request:

```
{
  "addressId": "665f51b9e5348a2f42b56c70",
  "paymentMethod": "online"
}
```

Example order response:

```
{
  "status": "success",
  "message": "Order created successfully",
  "data": {
    "order": {
      "_id": "665f51b9e5348a2f42b56c80",
      "buyerSnapshot": {
        "name": "Mohammed Hassan",
        "phone": "01012345678",
        "address": {
          "street": "15 Nile St",
          "city": "Cairo",
```

```
          "distanceMark": "Near mall entrance"
        }
      },
      "orderStatus": "waiting_store_acceptance",
      "payment": { "method": "online", "status": "pending" },
      "delivery": { "status": "none" },
      "totals": { "itemsTotal": 799, "deliveryFee": 0, "grandTotal":
        799 }
    }
  }
}
```

Delivery Module

Router file: `src/apis/order/router/delivery.router.ts`

All delivery endpoints use `authorized` and `isAuthorized(Role.FULFILLMENT, Role.ADMIN)`.

Delivery Endpoints

Endpoint	Description	Handler
`GET /delivery/orders`	List delivery orders visible to current fulfillment user/admin.	`getDeliveryOrdersHanc`
`PATCH /delivery/orders/:orderId/assign`	Assign order delivery rider.	`assignDeliveryOrderHa`
`PATCH /delivery/orders/:orderId/collect`	Mark delivery	`collectDeliveryOrderH`

Endpoint	Description	Handler
	collection started.	
`PATCH /delivery/orders/:orderId/start`	Mark order as on the way.	`startDeliveryOrderHar`
`PATCH /delivery/orders/:orderId/deliver`	Mark order delivered.	`deliverDeliveryOrderH`

Example delivery assignment request:

```
{
  "riderId": "665f51b9e5348a2f42b56c90"
}
```

Payments Module

Router file: `src/apis/order/router/payment.router.ts`

Payment Request Schema

Schema	Field	Type	Required	Validation / Rules	De
`PaymentWebhookBody`	`orderId`	string	Yes	Trimmed, min 1	No
`PaymentWebhookBody`	`event`	enum	Yes	`payment.succeeded`, `payment.failed`, `payment.refunded`	No

Payment Endpoint

Endpoint	Description	Handler	
`POST /payments/webhook`	Process payment provider event for an order.	`paymentWebhookHandler`	`src/apis/order webhook.contrc`

Example payment webhook request:

```
{
  "orderId": "665f51b9e5348a2f42b56c80",
  "event": "payment.succeeded"
}
```

Cross-Cutting Analysis

Detected Validation Libraries

Library / Mechanism	Location	Usage
Zod	`src/apis/**/validations/*.ts`	Body schemas and some controller-level param/query parsing.
Mongoose validation	`src/apis/**/models/*.ts`	Required fields, enums, min values, unique indexes, embedded documents.

Library / Mechanism	Location	Usage
Global quick validation	`src/middlewares/quick-validate.middleware.ts`	Email, password, token checks when matching field names exist.
XSS sanitizer	`src/middlewares/xss.ts`	Sanitizes strings in body/query/params.

Middleware Affecting Endpoints

Middleware	File	Scope
`express.json()`	`src/app.ts`	All routes.
`cookieParser()`	`src/app.ts`	All routes; required by refresh/logout.
`xss`	`src/middlewares/xss.ts`	All routes.
`helmet()`	`src/app.ts`	All routes.
`quickValidate`	`src/middlewares/quick-validate.middleware.ts`	All routes.
`authorized`	`src/middlewares/authentication.ts`	Protected routers/routes.
`isAuthorized`	`src/middlewares/authorized.ts`	Role-protected routes.
`validate`	`src/middlewares/validate-body.middleware.ts`	Routes with Zod body schemas.
`refreshLimiter`	`src/middlewares/auth-rate-limit.ts`	`POST /auth/refresh`.

Rate Limiting

Limiter	Endpoint(s)	Window	Max	Notes
`refreshLimiter`	`POST /auth/refresh`	15 minutes	100	Enabled.

Limiter	Endpoint(s)	Window	Max	Notes
`authSensitiveLimiter`	Intended auth-sensitive endpoints	15 minutes	5	Defined but commented out or register/login/logo

File Uploads

No active file upload middleware or multipart parser is used by the current `src/app.ts` router tree. Product/store image fields are string URLs/paths in JSON, not uploaded files.

Pagination, Sorting, Filtering

Resource	Pagination	Sc
Public products `/products`	`page`, `limit`, default `1`/`24`, max `100`	`pric` `pric` `newe` `best` `sell`
Store products	`page`, `limit`, default `1`/`24`, max `100`	New
Stores/categories/cart/wishlist/addresses/orders/delivery	Not paginated in current code	Fixe sorti whe impl

Caching

No response caching, Redis cache, HTTP cache headers, or cache middleware were detected in the live `src/` backend.

Security Notes

Area	Observation
JWT access tokens	Required as `Authorization: Bearer <token>` for protected routes.
Refresh tokens	Stored in HTTP-only, same-site strict cookies and hashed in `LoginSession`. Rotation and reuse detection are implemented.
Payment webhook	No signature verification middleware was found; consider validating provider signatures before mutating order payment state.
Route order	`GET /store/my-store` should be moved above `GET /store/:storeId` to avoid shadowing.
Param mismatch	`DELETE /store/:storeId/manage/delete` uses a route param named `storeId`, but `hardDeleteStoreHandler` reads `req.params.id`.
Validation gap	`reset-password` uses `newPassword` rather than `password`, so global `quickValidate` password rules do not apply to that field name.